Introduction to Programming II project log

Project title:	Coursework – Data Visualisation
Topic:	Week 13 and 14: Research and coding of heat map (population density)

What progress have you made on this topic?

Week 10-12

I added 2 major extensions to my project.

1. **Waffle Charts** – Global Warming Awareness by Age Group.

   I sourced survey data online that captured global warming awareness across different age groups. After exploring visualization options, I implemented two side-by-side waffle charts, each representing a different age group. Initially, both charts used the same color scheme, but I refined this by assigning distinct colors to each group. This improved visual contrast and made the comparison more intuitive for viewers.

2. **Circle Diagrams** – UK Food Safety Perceptions (2023 vs 2025)

   Using data extracted from an online article, I cleaned and formatted it into CSV for visualization. To compare changes over time, I designed circle diagrams where each category is represented by two concentric circles: the outer circle for 2025 and the inner for 2023. I encountered challenges with label readability, especially when text overflowed. To resolve this, I embedded HTML <div> elements inside the SVG using foreignObject, enabling automatic line wrapping, something native SVG <text> elements don't support.

Week 13-14

- **Heat map** for population density of East Asia countries

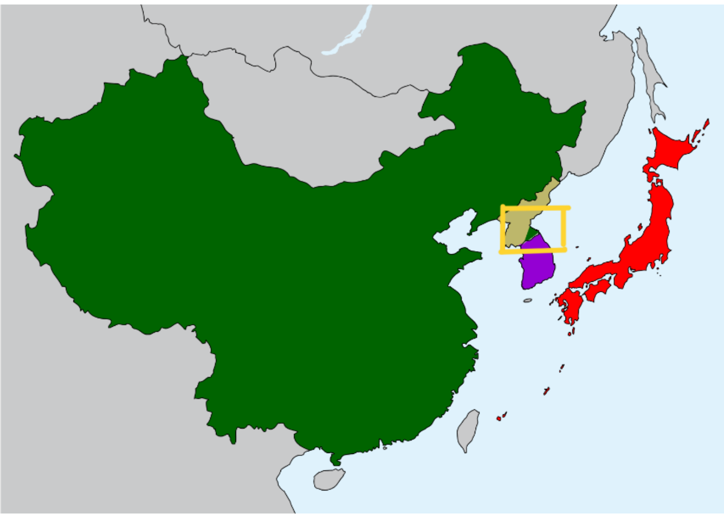
  I sourced population density data from a downloadable CSV and cleaned it to match the scope of my East Asia visualization. I then located an SVG map focused on East Asia and manually found and labelled the path data for each country. After integrating the SVG into my project, I applied heat map coloring principles to reflect density variations. To enhance usability, I implemented zoom and pan controls, along with hover interactions for dynamic feedback.

What problems have you faced and were you able to solve them?

There were several problems I faced when doing this extension.

1.Separating and Combining SVG Paths for East Asia Countries

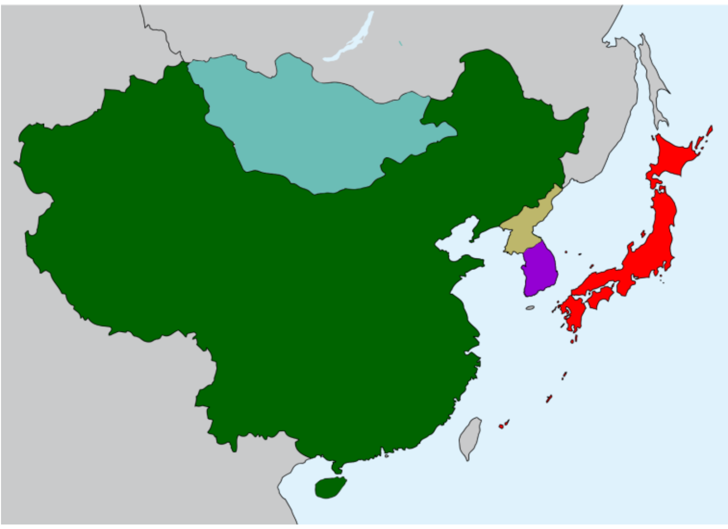
Problem: The original SVG map file contained an overwhelming number of paths, many of which were grouped or unlabeled. Notably, Korea was represented as a single combined path, making it impossible to manually separate North and South Korea due to the lack of distinct coordinate data. Additionally, countries like Mongolia were visually unified but internally fragmented into many paths with no label, thus unable to identify all paths for Mongolia. (Pic 1)



Pic 1: Original SVG map

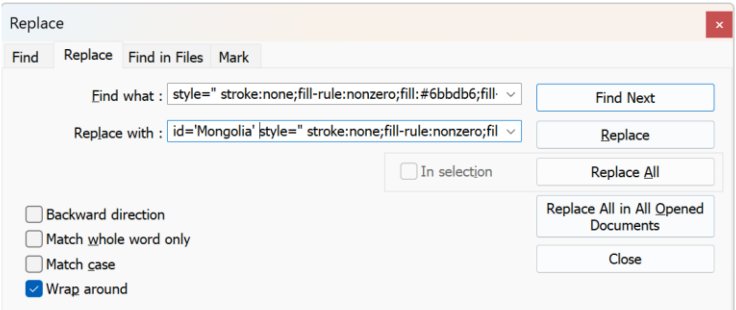
[Initial attempt to manually separate and combine paths. Inaccurate separation due to lack of distinct coordinates.]

Solution: To overcome this, I used a canvas-based tool with a smart pencil feature to manually color each East Asia country. (Pic 2) I then exported the result as an SVG file. This allowed me to visually distinguish countries by color and assign id attributes based on those color codes. However, this process also generated new path data based on the countries I colored, resulting in 456 separate paths for Mongolia alone. To efficiently label these paths, I used Ctrl+F and batch replace to search for each country's unique fill color and replace it with the corresponding id and full style declaration. (Pic 3)



Pic 2: Canvas-Coloured SVG map

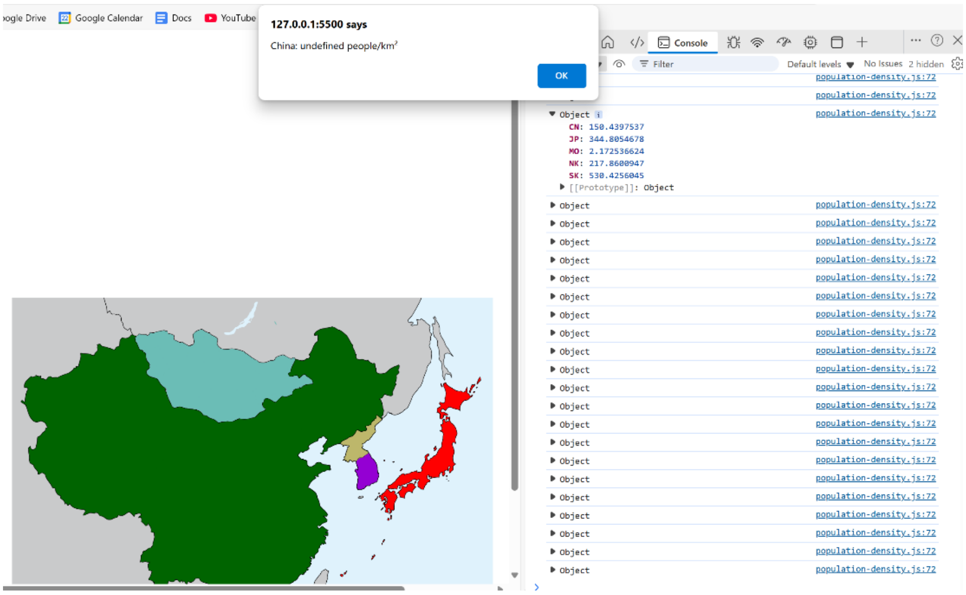
[Countries successfully colored, paths can be identified via colour search]



Pic 3: Batch Labeling via Find & Replace

2. Population Density Data not displayed properly.

Problem: The population density data for each country was not displaying correctly in the visualization. When triggered (alert), the value returned was undefined. (Pic 4)



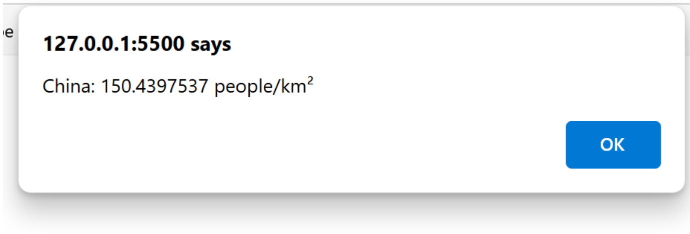
Pic 4: Alert Showing Undefined Data, Console Logs

Solution: I reviewed the console logs and inspected the object structure used to store density values. I realized that the keys in my object were based on country codenames (e.g., "KR") (Pic 4), while the alert logic was referencing full country names (e.g., "South Korea"). This mismatch caused the lookup to fail.

To resolve this, I restructured the object to use country names as keys, aligning it with the identifiers used in the alert and SVG path IDs. (Pic 5) After making this change, the data displayed correctly. (Pic 6)

```
const country = table.getString(i, 'Country Name');
const density = table.getNum(i, 'Population density');
self.densityData[country] = density;
```

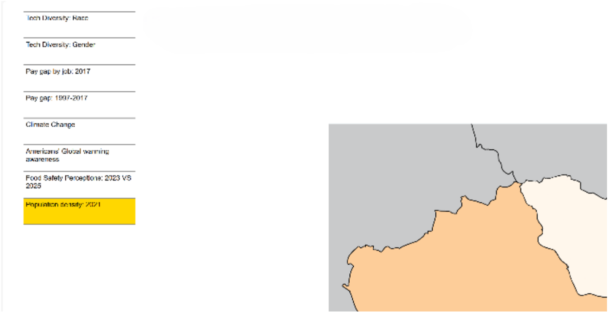
Pic 5: Code – Object Assignment Using Country Names

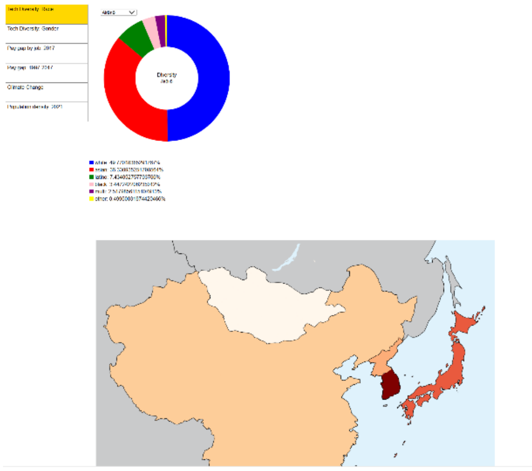


Pic 6: Corrected Alert Output

3. Map Placement and Zoom Behavior Issues

Problem: As shown in Pic 7, the map was misaligned. It was slightly below the menu instead of being beside it, making the interface appear cluttered. I had scaled the map to 60% of its original size, but when the user initiates a zoom or pan, the map would first expand back to its original size. Only after a second interaction would it begin adjusting correctly. Additionally, the map remained visible even when switching to other visualizations. (Pic 8)



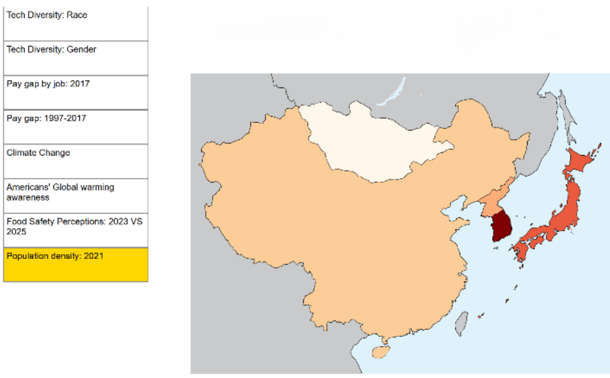


Pic 7: Misaligned Map Placement

Pic 8: Zoom Issue

Solution: To resolve this, I defined a fixed transform (initialTransform) with both scale and translation values, and applied it when initializing d3.zoom(). (Pic 9) This ensures that the map loads at the correct size and position, and prevents it from snapping back to full size on first interaction. I also added a destroy function to clean up the map when switching views, which resolved the issue of it lingering across visualizations. (Pic 10)

```
//Initial positioning
const initialTransform = d3.zoomIdentity
  .translate(-90, -10)
  .scale(0.6);
mapGroup.attr('transform', initialTransform);
//Add zoom behavior
svg.call(
  d3.zoom()
    .on('zoom', function (event) {
      mapGroup.attr('transform', event.transform);
    })
).call(
  d3.zoom().transform, initialTransform
);
```

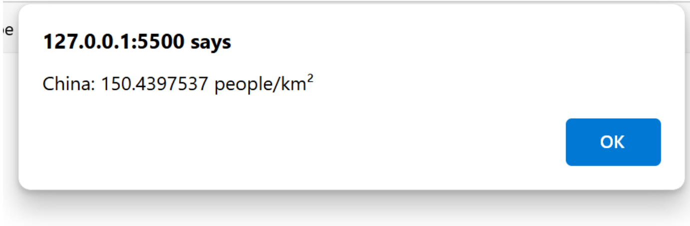


Pic 9: Code – initialTransform setup

Pic 10: Final Map Layout with legend space on top

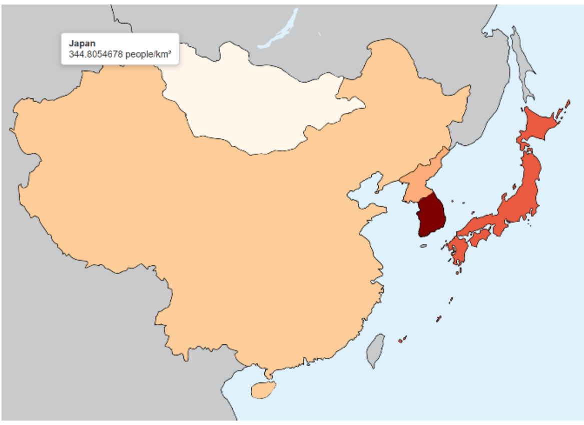
4. Disruptive Data Display Using alert()

Problem: I used alert() to display the country's population density data when users click on a country. While this worked functionally, it quickly became disruptive, especially when exploring multiple countries, since each alert has to be dismissed manually. This breaks the flow of interaction and creates a frustrating user experience. (Pic 11)



Pic 11: Alert box Displaying Population Density

Solution: I replaced the click-based alerts with a hover interaction. Now, when users hover over a country, its data appears in a display box near the cursor. This approach is smoother and more intuitive, allowing users to explore the map without interruptions.
However, I encountered a positioning issue. The display box appeared far off to the left when hovering over Japan (Pics 12, 14). This was due to relying on relative coordinates affected by SVG transforms. To fix this, I updated the code to use event.pageX and event.pageY (Pic 15), which provide absolute screen coordinates and are unaffected by SVG transformations. This ensures consistent and accurate tooltip placement across all countries. (Pic 13)



Pic 12: Tooltip Misaligned on Hover

Pic 13: Tooltip Correctly Positioned

```
.on('mousemove', (event) => {
  const [x, y] = d3.pointer(event);
  tooltip
    .style('left', `${x + 20}px`)
    .style('top', `${y}px`);
})
```

Pic 14: Code – Incorrect tooltip positioning

```
.on('mousemove', (event) => {
  tooltip
    .style('left', `${event.pageX + 20}px`)
    .style('top', `${event.pageY}px`);
})
```

Pic 15: Code – Incorrect Tooltip Positioning

What are you planning to do over the next few weeks?

Following the Gantt chart I created earlier, my focus over the next few weeks will be on finalizing the heat map component. This includes:

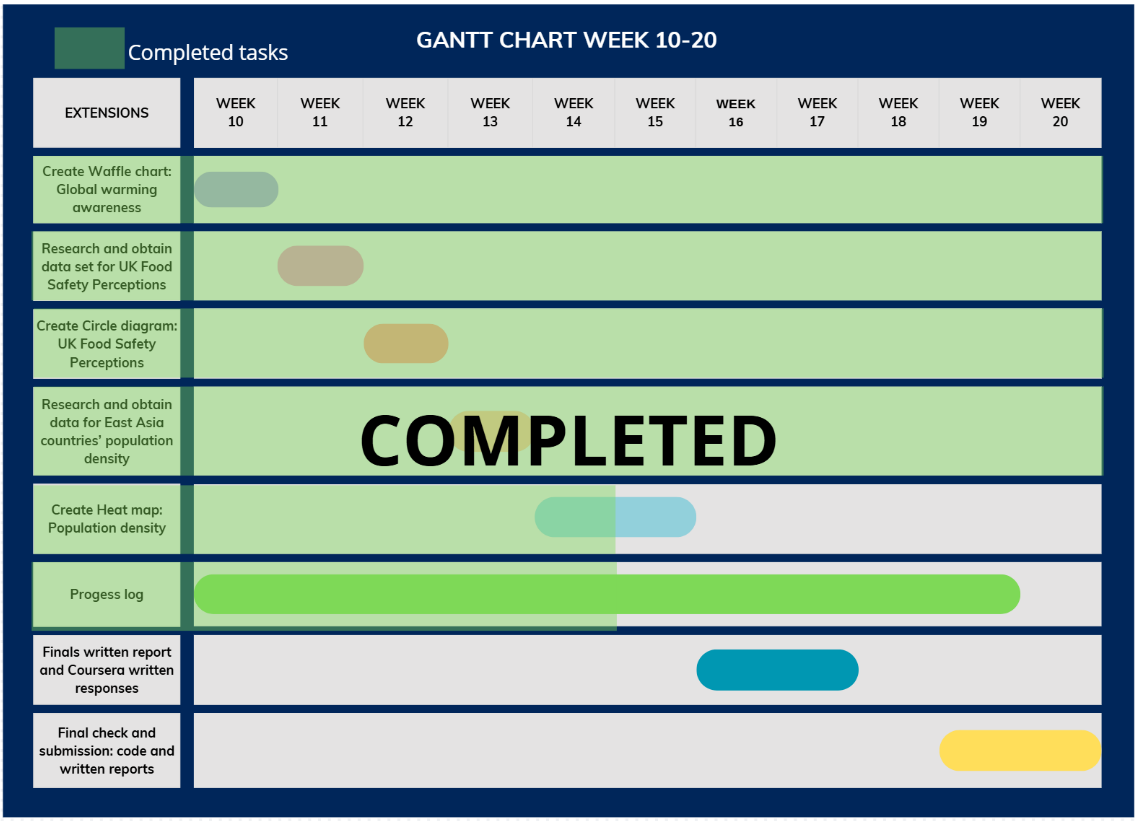
- **Adding a legend** to clearly communicate the population density scale and improve interpretability.
- **Debugging and feature polishing**, with a focus on identifying hidden errors or rendering issues that may affect smooth execution.
- **Refining interactions and layout**, ensuring that hover effects, zoom/pan controls, and color transitions behave consistently.

This phase will help ensure the visualization is not only technically sound but also user-friendly and visually coherent.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

I am on target to successfully complete my project. Most of the key extensions, including the waffle chart, circle diagram, and core structure of the heat map, have been implemented. The remaining tasks involve finalising minor components of the heat map and conducting thorough checks.

Throughout development, I've performed incremental testing to ensure each extension runs smoothly and that all code behaves as expected. My next steps will focus on comprehensive debugging and final validation to ensure the entire project is stable, visually coherent, and free of critical issues.

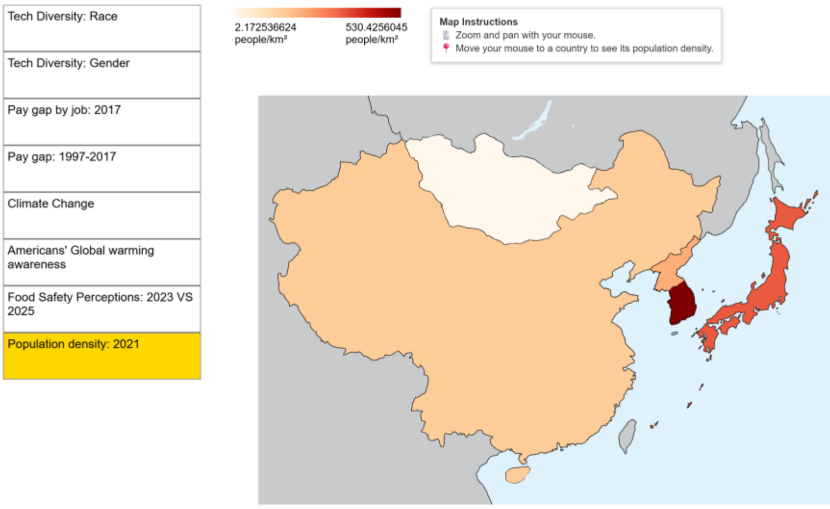


Introduction to Programming II project log

Project title:	Coursework – Data Visualisation
Topic:	Week 15 and 16: Complete Heat Map and Feature Polishing

What progress have you made on this topic?

I added a legend and map instructions to the population density visualization to improve readability. I also polished several features, including resizing the waffle charts and repositioning the map instructions for better layout and clarity.

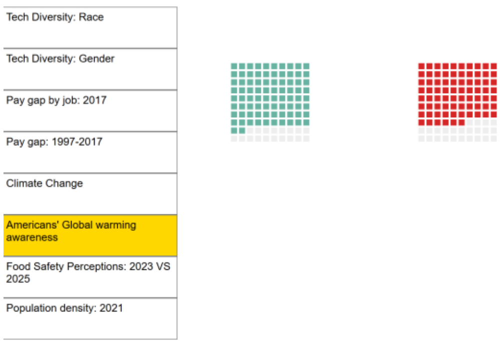


Pic 1: Legend and Map Instructions added

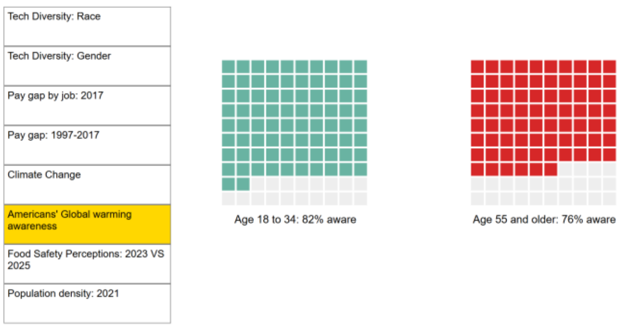
<u>Feature polishing</u>

1. **Enlarging the Waffle Charts**

   To increase visual clarity, I increased the size of the waffle charts.



Pic 2: Initial Waffle Charts' size



Pic 3: Enlarged Waffle Charts

```
const squareSize = 9;
const gap = 2;
```

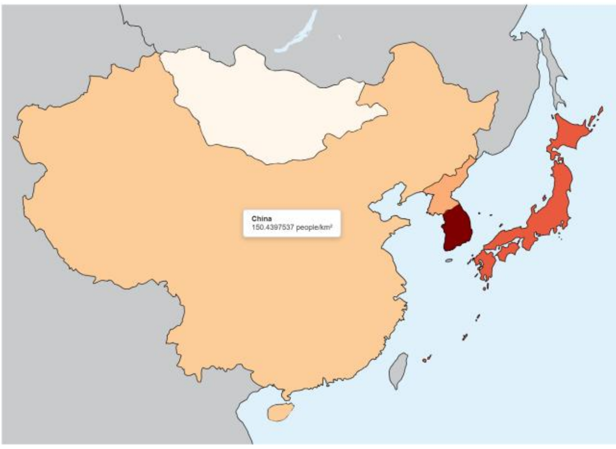
Pic 4: Code – Initial size setting

```
const squareSize = 20;
const gap = 3;
```

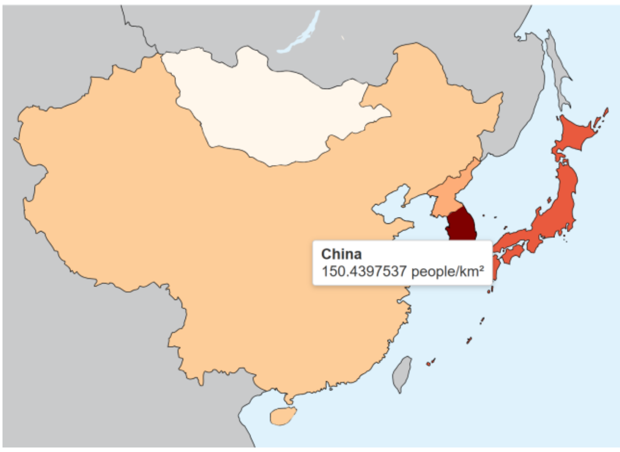
Pic 4: Code – Final size setting

2. **Enlarging Hover Interaction – Heat Map**

I increased the font size of the hover tooltip from 9px to 15px to make data easier to read and improve accessibility.



Pic 4: Initial font size of Hover Interaction

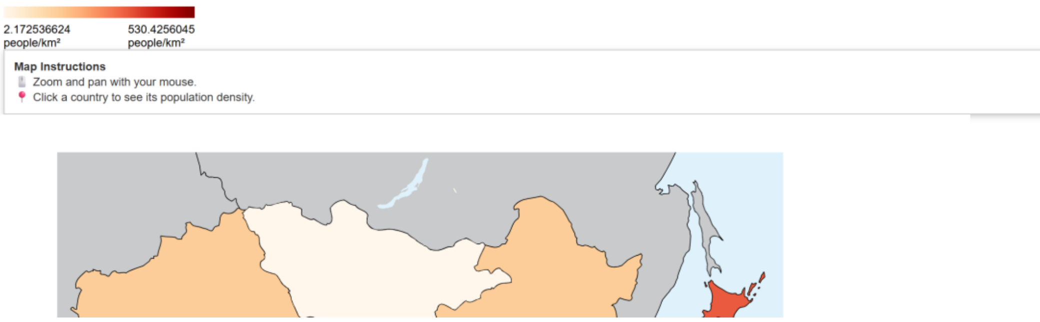


Pic 5: Final font size of Hover Interaction

What problems have you faced and were you able to solve them?

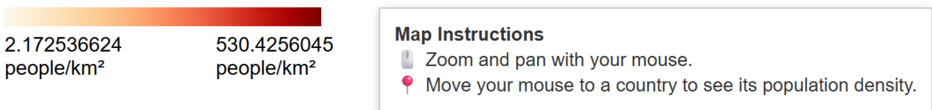
1. Map Instructions – wrong positioning

Problem: The Map Instructions were intended to be beside the legend. However, it appeared below the legend instead, and had a very long box (Pic 6)



Pic 6: Map Instructions not positioned as intended

Solution: I added a legendWrapper variable to control alignment and position the Map Instructions as intended.



Pic 7: Map Instructions aligned

```
const legendWrapper = d3.select('#heat-map')
  .append('div')
  .attr('class', 'legend')
  .style('display', 'legend-wrapper')
  .style('display', 'flex')
  .style('align-items', 'flex-start')
  .style('gap', '40px')
  .style('margin-left', '45px')
  .style('margin-top', '20px')
  .style('font-size', '14px')
  .style('padding-bottom', '4px');

const legend = legendWrapper
  .append('div')
  .attr('class', 'legend')
```

```
//instructions
legendWrapper.append('div')
  .attr('class', 'map-instructions')
  .style('background', '#fff')
  .style('border', '1px solid #ccc')
  .style('padding', '10px')
  .style('font-size', '13px')
  .style('color', '#333')
  .style('box-shadow', '0 2px 6px rgba(0,0,0,0.2)')
  .html(`
    <strong>Map Instructions</strong><br>
    🖱️ Zoom and pan with your mouse.<br>
    📍 Move your mouse to a country to see its population density.
  `);
```

Pics 8 & 9: Code – Usage of legendWrapper to align Map Instructions

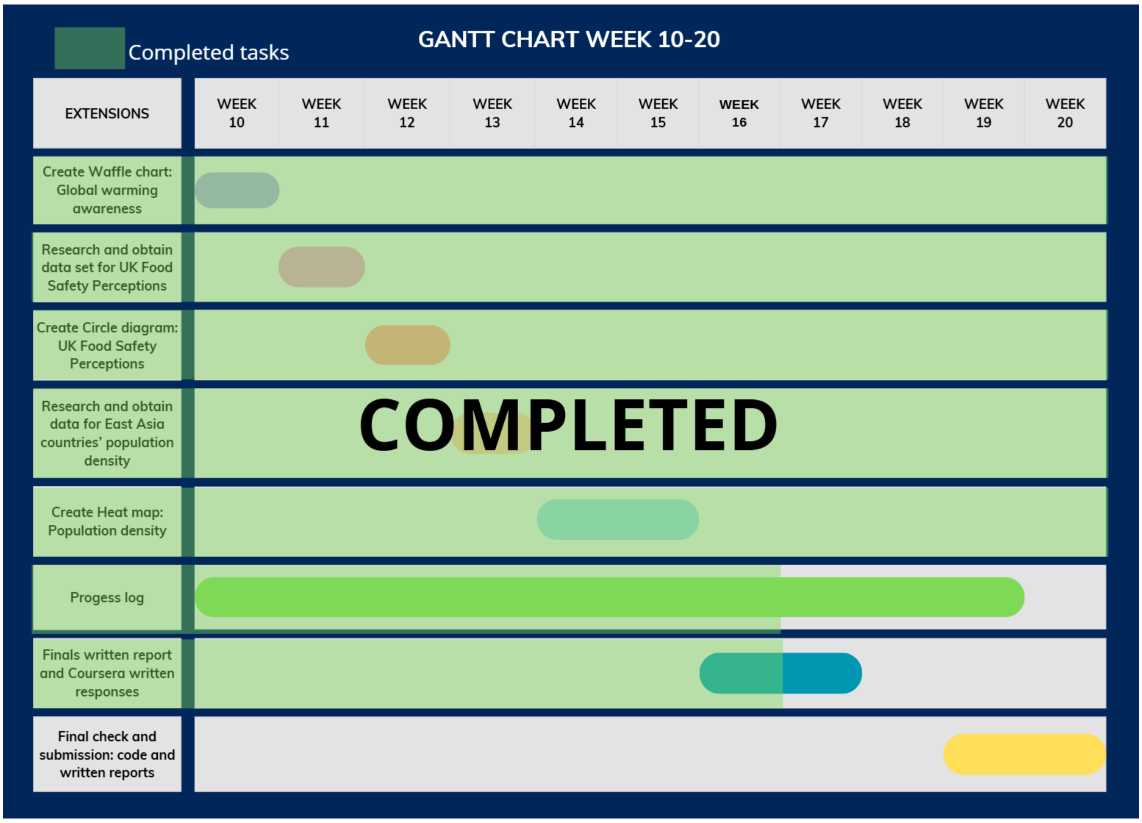
What are you planning to do over the next few weeks?

Over the next few weeks, my focus will be on:

- **Debugging** to ensure that the newly added legend, map instructions, and polished features do not introduce any errors.
- **Stability and usability testing** to validate performance and user experience across different scenarios.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

I am on target to successfully complete my project. All planned extensions have been implemented, and minor checks have been conducted during development. No major bugs or errors have been identified so far. Once I complete a final general review, I will proceed with stability and usability testing to finalize the project.


Completed tasks
GANTT CHART WEEK 10-20
EXTENSIONS
WEEK 10
WEEK 11
WEEK 12
WEEK 13
WEEK 14
WEEK 15
WEEK 16
WEEK 17
WEEK 18
WEEK 19
WEEK 20
Create Waffle chart: Global warming awareness
Research and obtain data set for UK Food Safety Perceptions
Create Circle diagram: UK Food Safety Perceptions
Research and obtain data for East Asia countries' population density
COMPLETED
Create Heat map: Population density
Progess log
Finals written report and Coursera written responses
Final check and submission: code and written reports

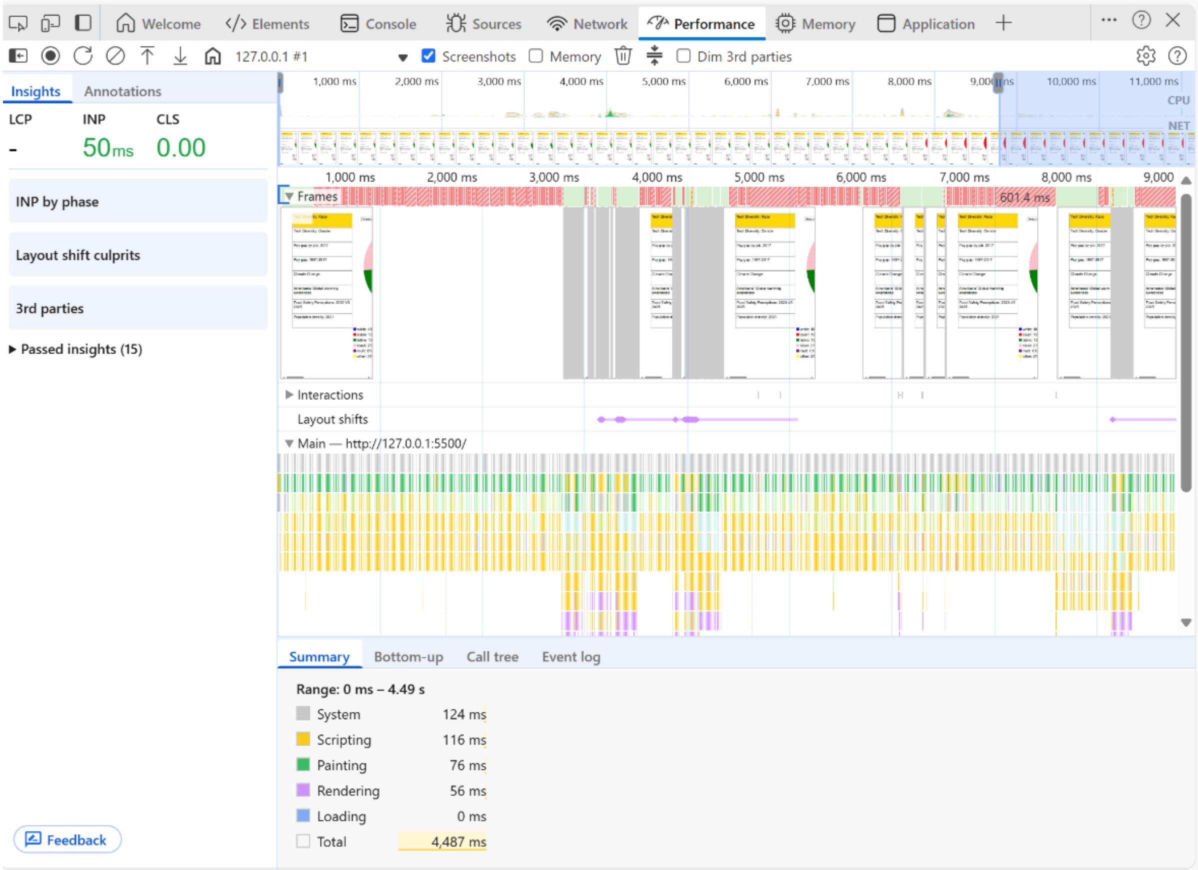
Introduction to Programming II project log

Project title:	Coursework – Data Visualisation
Topic:	Week 17 and 18: Debugging and Stability tests

What progress have you made on this topic?

I inspected each Data Visualisation and checked their performance.

<u>Tech Diversity: Race – Donut Chart</u>



Performance:

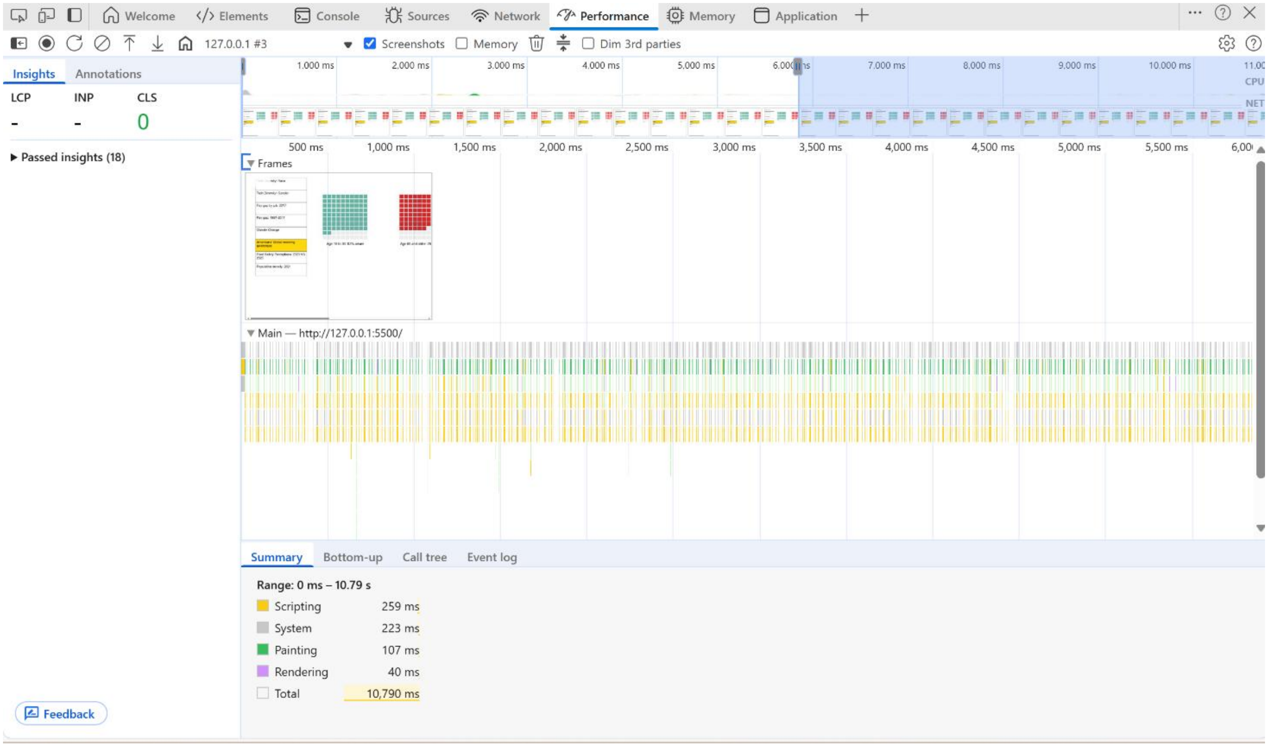
INP: 50ms → Dashboard responds quickly to user interactions

CLS: 0.00 → No layout shifts, ensures stable rendering and good UX

Rendering Efficiency:

Scripting, painting, and rendering times are all below 150 ms, indicating optimised SVG and D3.js rendering. This ensures smooth frame progression, responsive JavaScript execution, and lag-free visual updates.

American's Global Warming awareness – Waffle Charts

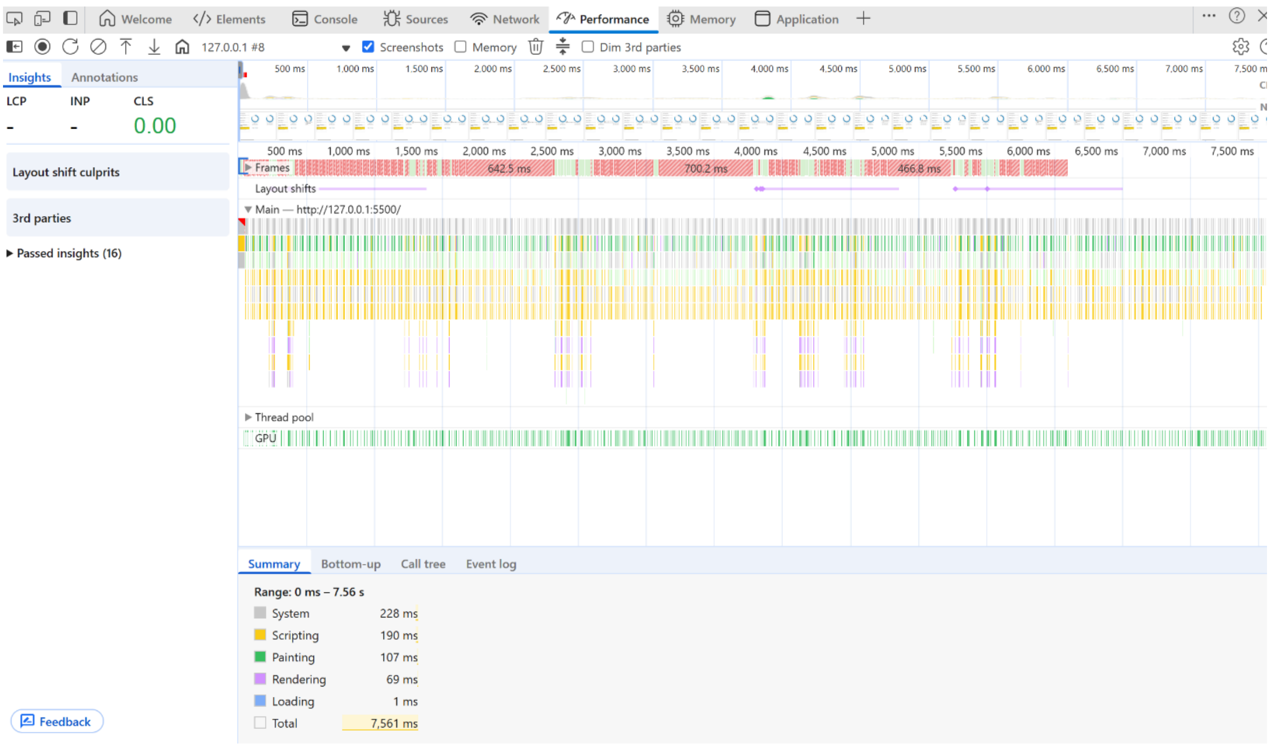


Rendering Efficiency:

Scripting: 259ms → Within acceptable range
Painting: 107ms → Smooth rendering of SVG elements, no visual lag detected.
Rendering: 40ms → Highly efficient layout and style recalculations.

These metrics confirm that the waffle charts perform reliably under real-world conditions, with no jank or layout instability. The visualisation remains responsive and accessible.

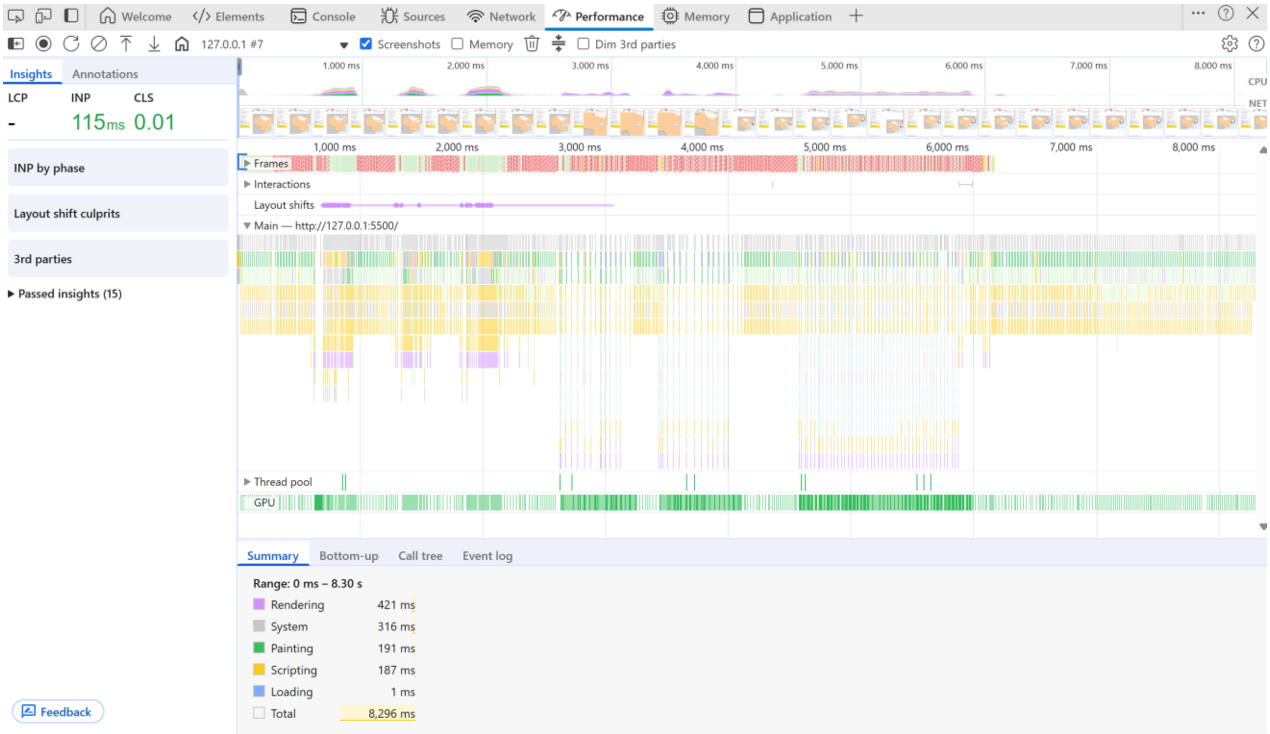
Food Safety Perceptions: 2023 VS 2025 – Circle Diagram



Rendering Efficiency:

Scripting: 190ms → Efficient JS execution, suggests optimised data parsing and event handling.
Painting: 107ms → Smooth rendering of SVG elements, no visual lag detected.
Rendering: 69ms → Highly efficient layout and style recalculations.

<u>Population Density: 2021 – Heat Map</u>



Performance:

INP: 115ms → Dashboard responds quickly to user interactions
CLS: 0.01 → No noticeable layout shifts, ensures stable rendering and smooth UX.

Rendering Efficiency:

Scripting: 187ms → Efficient JS execution, suggests optimised data parsing and event handling.
Painting: 191ms → Smooth rendering of SVG elements, no visual lag detected.
Rendering: 421ms → Moderate complexity

Summary

All feature enhancements and polishing efforts were validated through targeted debugging and performance testing. Metrics such as INP, CLS, scripting, rendering, and painting times remain within optimal ranges, confirming that recent changes did not introduce regressions. The dashboard continues to perform responsively and render stably, ensuring a smooth and accessible user experience.

What problems have you faced and were you able to solve them?

No critical issues were encountered during development. However, extensive debugging and stability testing were conducted to proactively ensure that new features, such as chart resizing, legend positioning, and tooltip behaviour, did not introduce regressions.
Performance audits confirmed that all metrics remained within optimal ranges, and layout integrity was preserved across visualizations

What are you planning to do over the next few weeks?

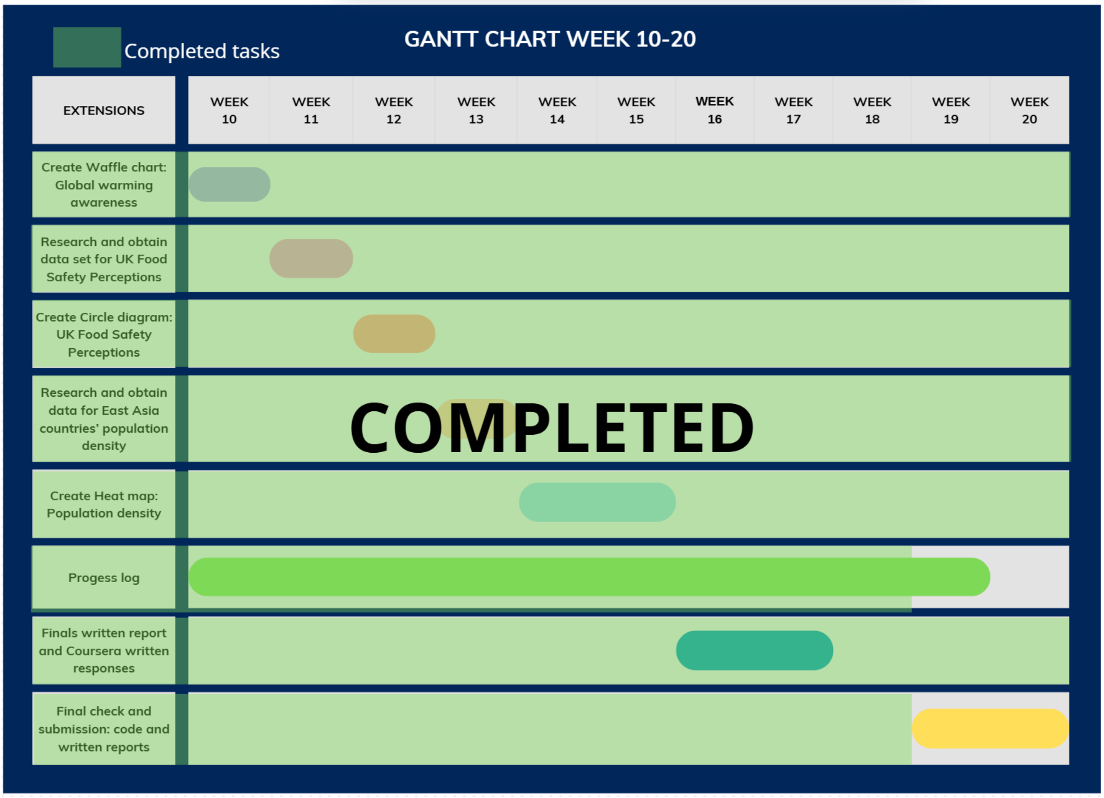
I plan to conduct additional stability testing to ensure that all visualizations remain performant and error-free across interactions.
I'll also begin usability testing to validate the overall user experience, focusing on clarity, accessibility, and intuitive navigation. These tests will form the final review and verification stage before submission, confirming that the dashboard is both technically sound and user-friendly.

Plan:

1. Go through all the visualisations to check for possible errors (eg. Glitch, visualisation not displaying)
2. Conduct a survey, the System Usability Scale (SUS), to 3 people, to test my data visualisations

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

I am on target to successfully complete my project. No issues have been identified at this stage of debugging and testing. Assuming continued stability, my project will be ready for submission after the final round of performance and usability testing.



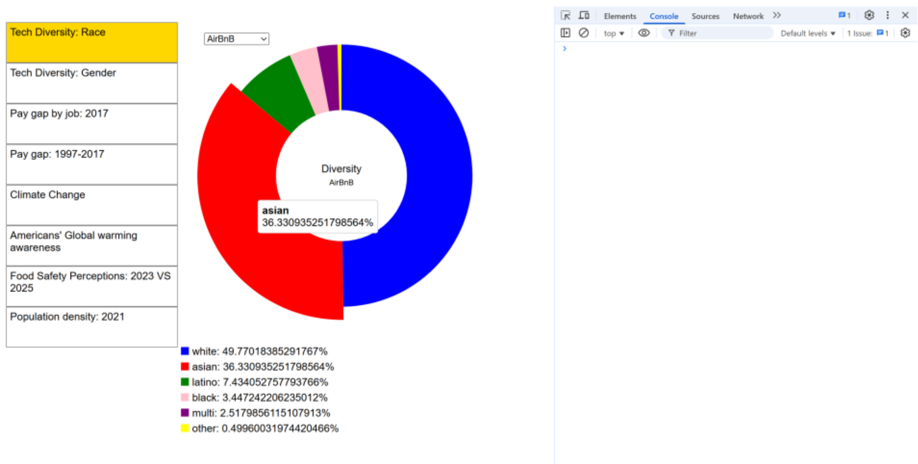
Introduction to Programming II project log

Project title:	Coursework – Data Visualisation
Topic:	Week 19 and 20: Usability tests

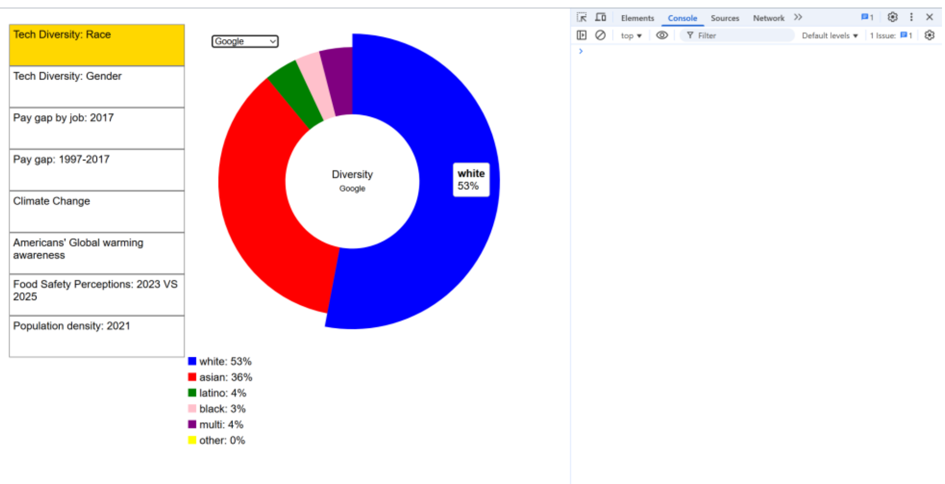
What progress have you made on this topic?

Stability test: I went through all data visualisations to check for consistent performance.

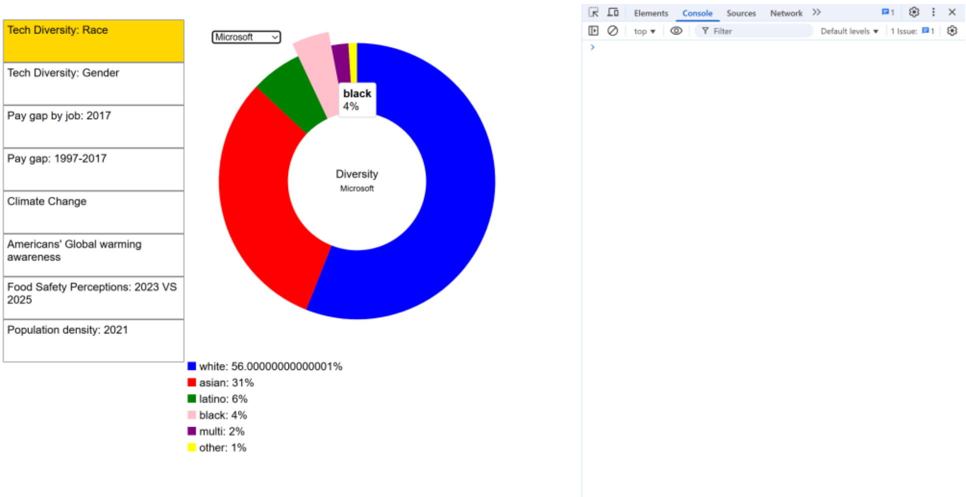
1. Tech Diversity: Race



Pic 1: Company: AirBnB



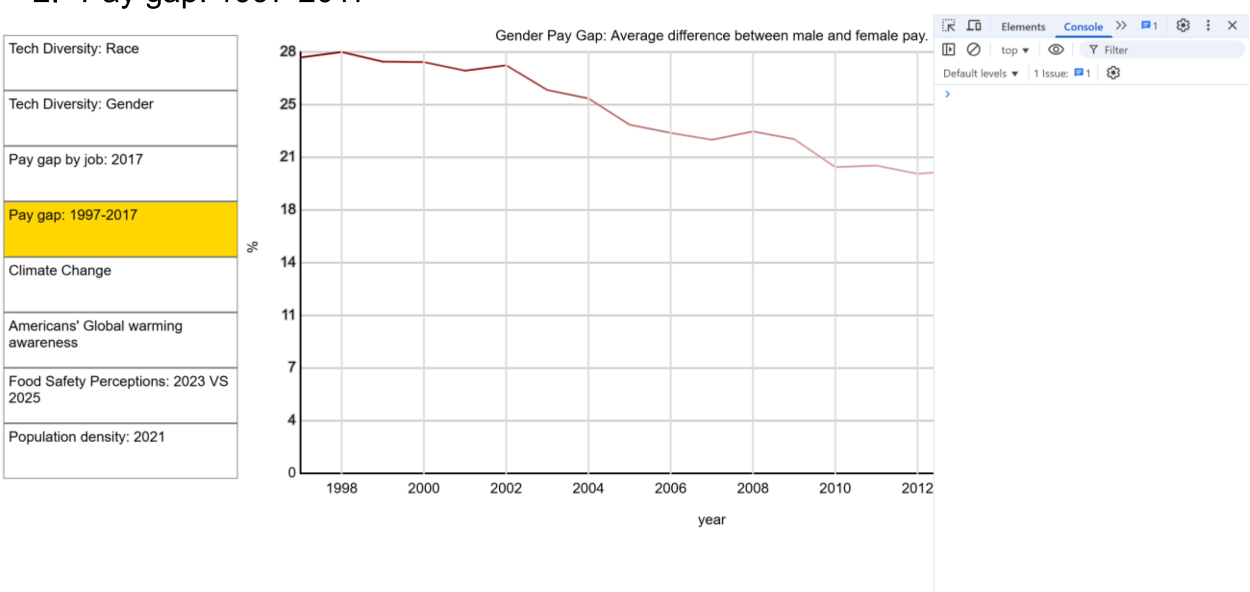
Pic 2: Company: Google



Pic 3: Company: Microsoft

CHECKLIST	
✓	Chart renders correctly on load
✓	Legends are positioned and readable
✓	No console errors or warnings
✓	Tooltips, hover states, or interactivity work as intended
✓	Performance remains smooth

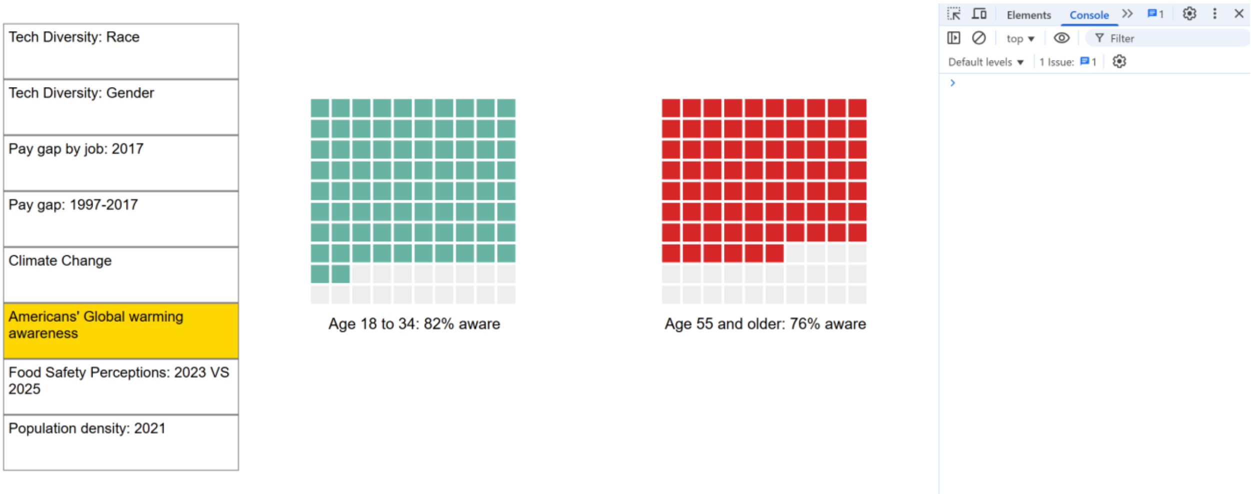
2. Pay gap: 1997-2017



Pic 4: Colour intensity line graph

CHECKLIST	
✓	Graph renders correctly on load
✓	No console errors or warnings
✓	Colour intensity work as intended
✓	Performance remains smooth

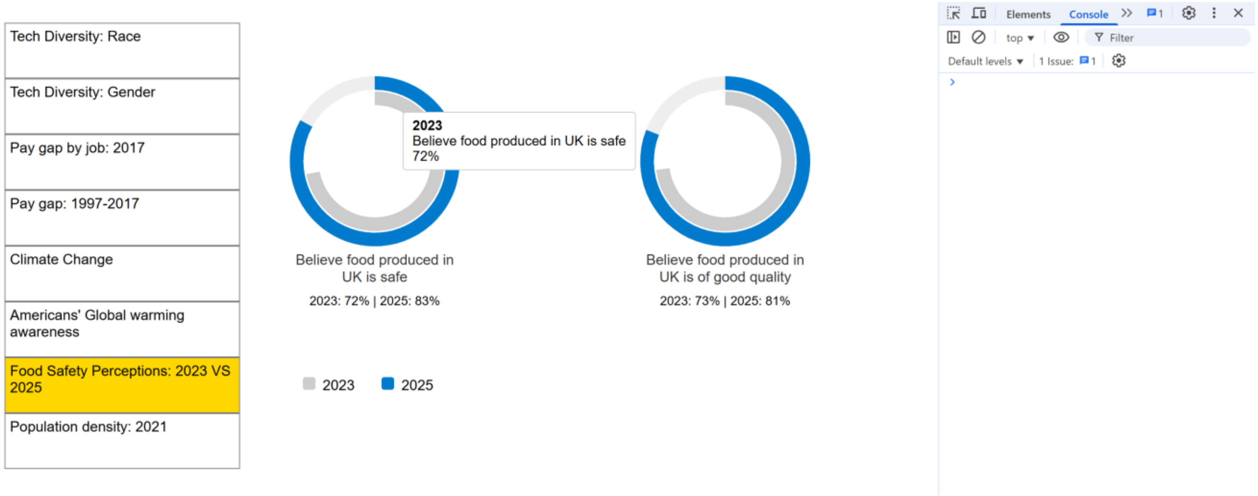
3. American’s Global warming awareness



Pic 5: Waffle Charts

CHECKLIST	
✓	Graph renders correctly on load
✓	No console errors or warnings
✓	Legends are positioned and readable
✓	Performance remains smooth

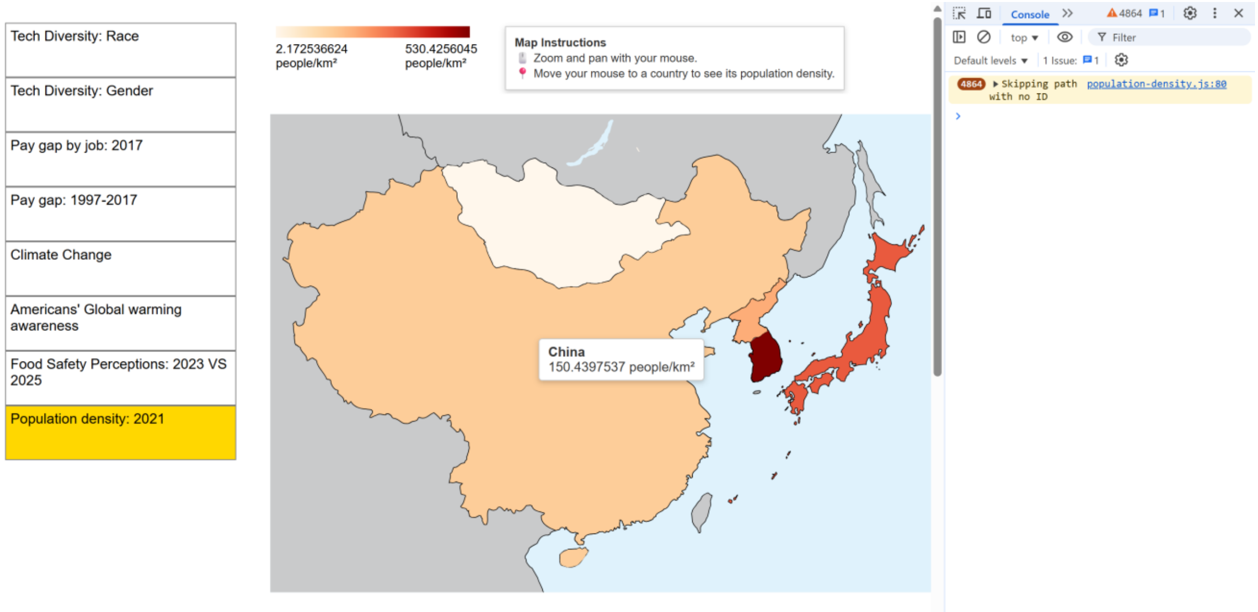
4. Food Safety Perception: 2023 VS 2025



Pic 6: Circle Diagrams

CHECKLIST	
✓	Chart renders correctly on load
✓	Legends are positioned and readable
✓	No console errors or warnings
✓	Tooltips, hover states, or interactivity work as intended
✓	Performance remains smooth

5. Population density: 2021

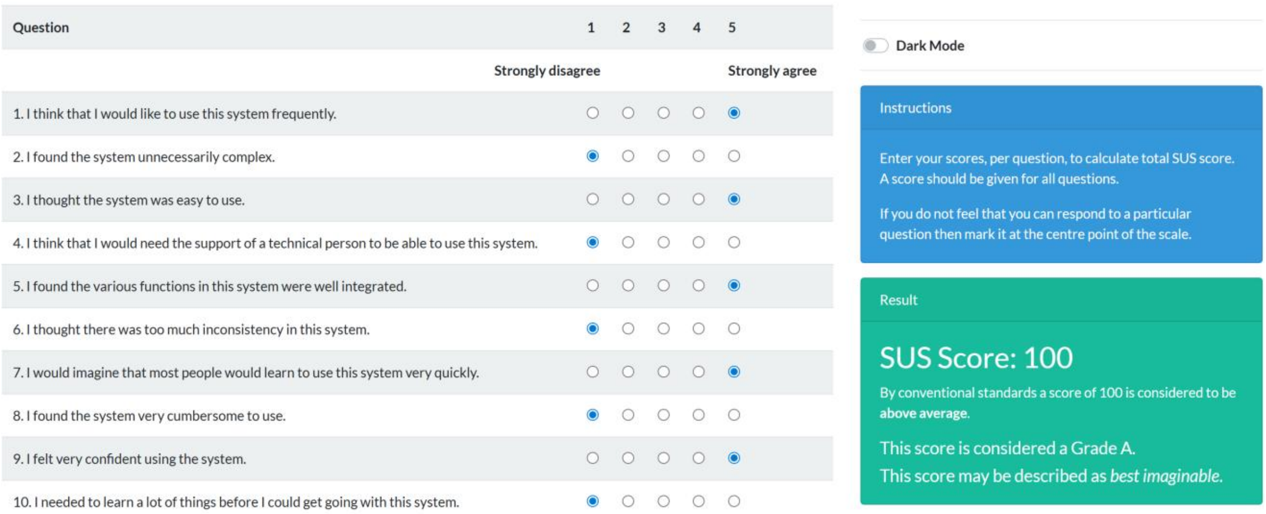


Pic 7: Heat Map

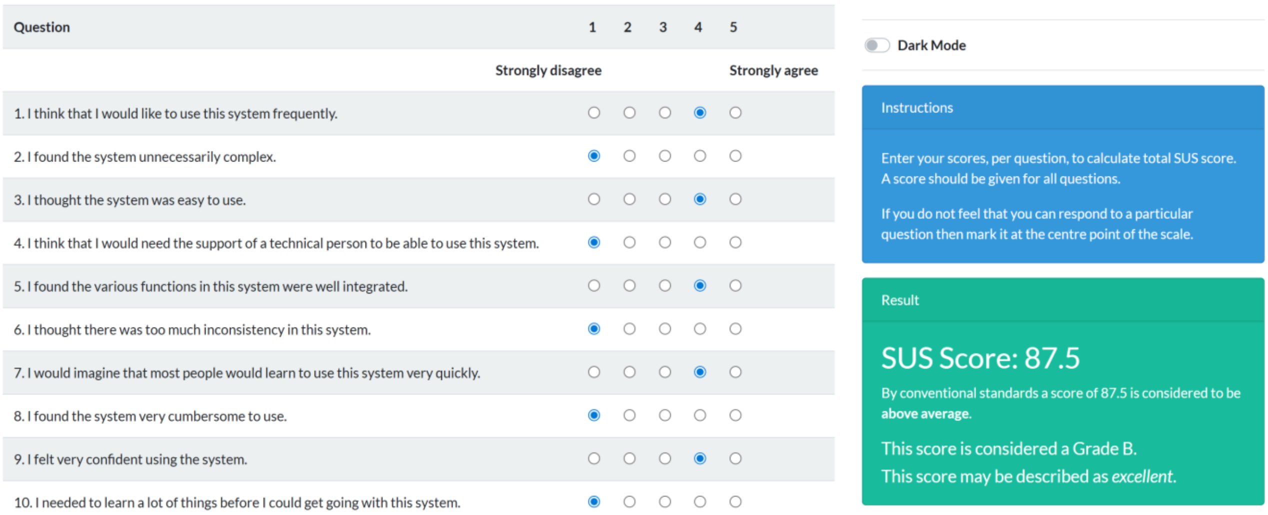
CHECKLIST	
✓	Chart renders correctly on load
✓	Legends and instructions are positioned and readable
✓	No console errors or warnings
✓	Tooltips, hover states, or interactivity work as intended
✓	Performance remains smooth

All extended data visualisations functioned as intended, with no errors detected.

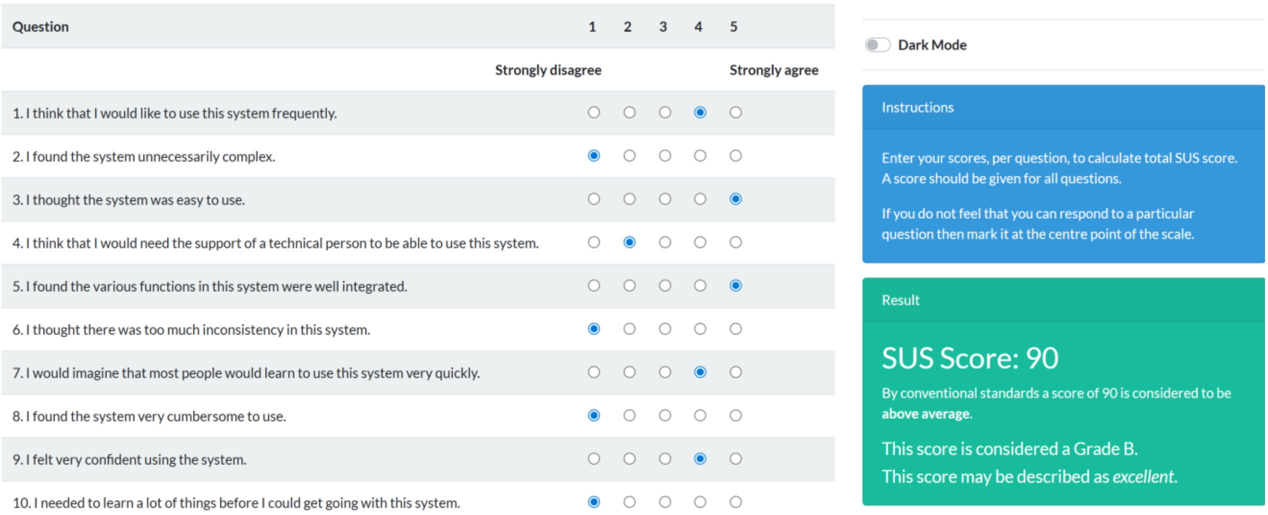
I also conducted usability surveys with three participants using the System Usability Scale (SUS). Their responses provided valuable insights for the system's overall usability and user experience.



Pic 8: Response 1



Pic 9: Response 2



Pic 10: Response 3

All 3 responses had SUS scores above average, indicating a generally positive perception of the system's usability. This suggests that the system is perceived as usable and well-designed by its target users.

What problems have you faced and were you able to solve them?

No critical issues were encountered. All extensions and feature enhancements were successfully integrated and validated through stability and usability testing.

What are you planning to do over the next few weeks?

In this final week, I'm consolidating my work, refining documentation, and ensuring all deliverables meet quality standards.

Are you on target to successfully complete your project? If you aren't on target, how will you address the issue?

I am on target to successfully complete my project. Following a comprehensive round of stability, usability, and performance testing, all features have been validated and the final version is ready for submission.